

Final Project Report

Software Quality Assurance [MPCS 56540]

Fatima Madey

05/29/2026

<https://github.com/fatimamadey/mpcs-sqa/tree/main/final-project>

Q1: Linter / Static Review

I chose Flake8 as the linter for this project. In Assignment 1, I used Pylint. And I wanted to switch it up. Pylint produces a high volume of opinionated warnings that may or may not be what I am looking for. If it is not, Flake8 is a better choice. It wraps three tools into a single interface: `pycodestyle` for PEP 8 style enforcement, `pyflakes` for logical errors such as unused imports and undefined names, and `mccabe` for complexity analysis. I also enabled the `flake8-bugbear` plugin, which adds a curated set of additional checks for common bug patterns that the base tools miss.

The configuration is committed to the repo at `.flake8`:

<https://github.com/fatimamadey/mpcs-sqa/blob/main/final-project/.flake8>

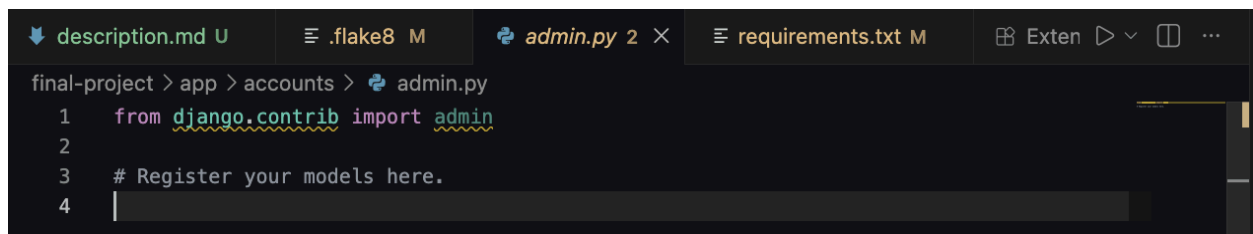
Can run linter with:

Shell

```
cd final-project & flake8 app/ --config .flake8
```

Example 1 - Imported but Unused [F401]

Original Code:



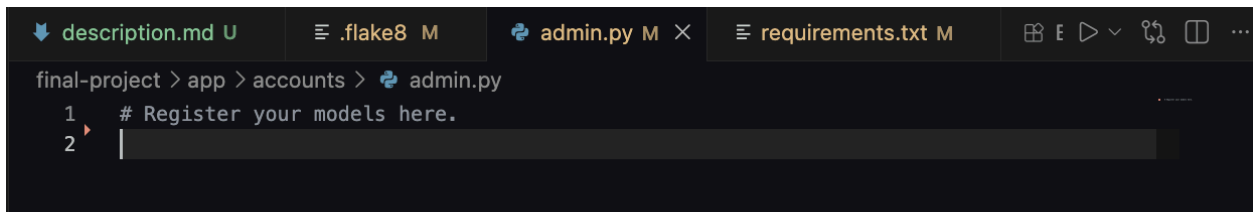
```
final-project > app > accounts > admin.py
1  from django.contrib import admin
2
3  # Register your models here.
4  |
```

Linter Warning:

Python

```
./accounts/admin.py:1:1: F401 'django.contrib.admin' imported but unused
```

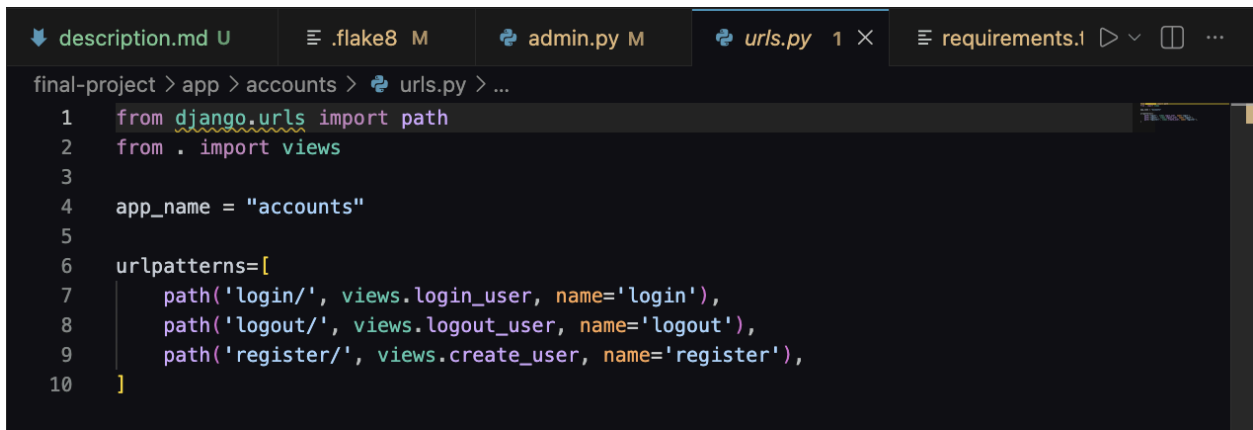
Corrected Code: [delete line 1]



```
description.md U .flake8 M admin.py M requirements.txt M
final-project > app > accounts > admin.py
1 # Register your models here.
2 |
```

Example 2 - Missing Whitespace Around Operator [E225]

Original Code:

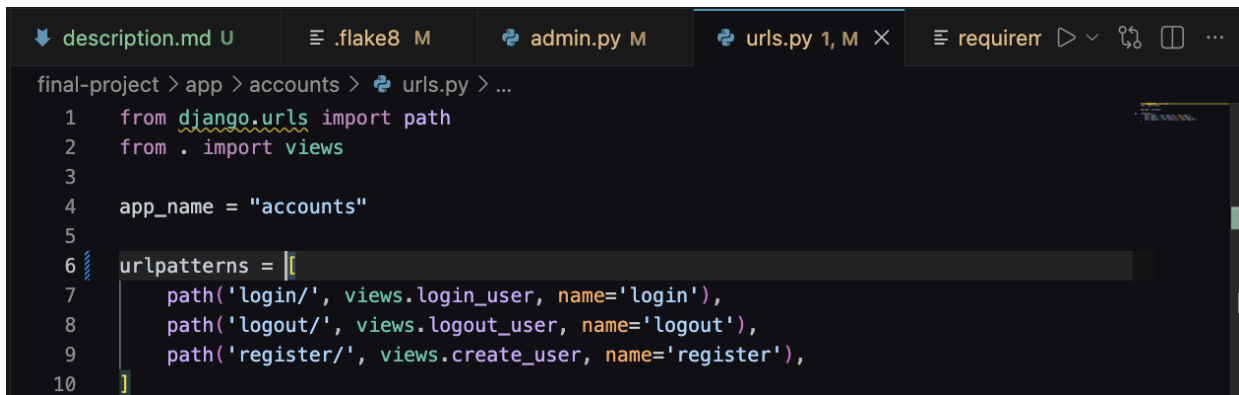


```
description.md U .flake8 M admin.py M urls.py 1 X requirements.1
final-project > app > accounts > urls.py > ...
1 from django.urls import path
2 from . import views
3
4 app_name = "accounts"
5
6 urlpatterns=[
7     path('login/', views.login_user, name='login'),
8     path('logout/', views.logout_user, name='logout'),
9     path('register/', views.create_user, name='register'),
10 ]
```

Linting Warning:

Python
./accounts/urls.py:6:12: E225 missing whitespace around operator

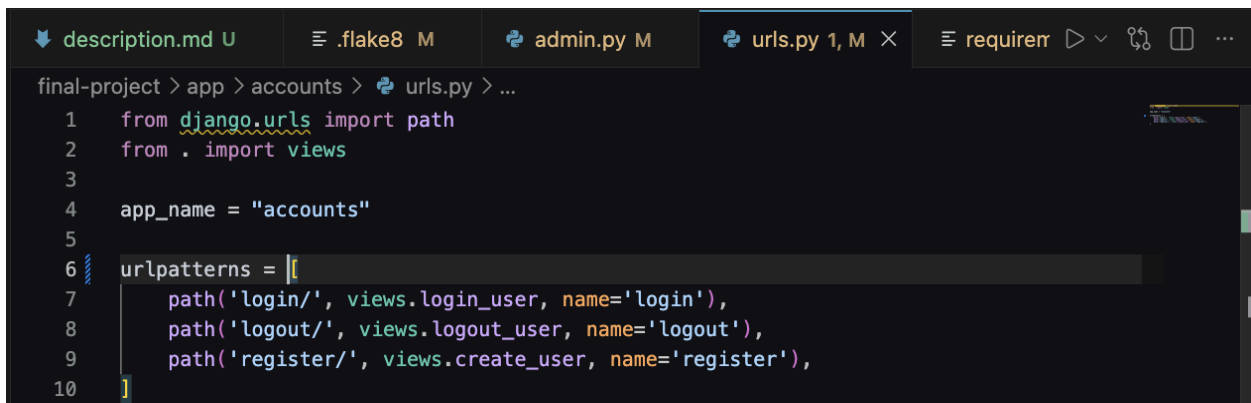
Corrected Code:



```
description.md U .flake8 M admin.py M urls.py 1, M X requirem
final-project > app > accounts > urls.py > ...
1 from django.urls import path
2 from . import views
3
4 app_name = "accounts"
5
6 urlpatterns = [
7     path('login/', views.login_user, name='login'),
8     path('logout/', views.logout_user, name='logout'),
9     path('register/', views.create_user, name='register'),
10 ]
```

Example 3 - No Newline at End of File [W292]

Original Code:



The screenshot shows a code editor with a dark theme. The file explorer at the top shows 'description.md', '.flake8', 'admin.py', 'urls.py', and 'requirements'. The code in 'urls.py' is as follows:

```
1 from django.urls import path
2 from . import views
3
4 app_name = "accounts"
5
6 urlpatterns = [
7     path('login/', views.login_user, name='login'),
8     path('logout/', views.logout_user, name='logout'),
9     path('register/', views.create_user, name='register'),
10 ]
```

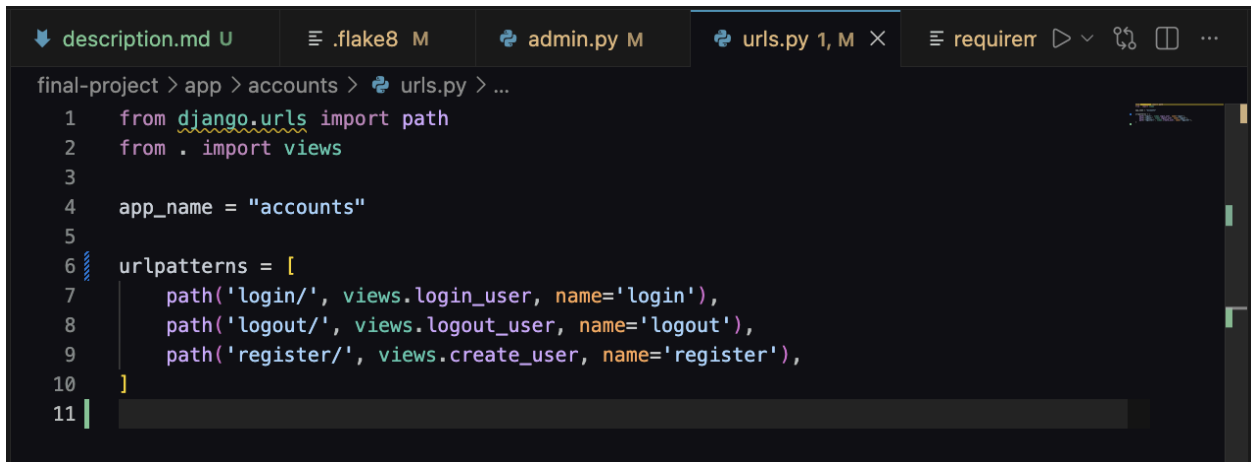
The cursor is at the end of line 10. A warning icon is visible on line 6.

Linting Warning:

Python

./accounts/urls.py:10:2: W292 no newline at end of file

Corrected Code:



The screenshot shows the same code editor as before, but the code in 'urls.py' now has a newline at the end of the file:

```
1 from django.urls import path
2 from . import views
3
4 app_name = "accounts"
5
6 urlpatterns = [
7     path('login/', views.login_user, name='login'),
8     path('logout/', views.logout_user, name='logout'),
9     path('register/', views.create_user, name='register'),
10 ]
11
```

The cursor is now at the beginning of line 11, indicating a new line has been added.

Example 4 - Line too Long [E501]

Original Code:

```
description.md U .flake8 M admin.py M views.py 6 X urls.py 1, M requirements.txt M Extension: GitHub Actions
final-project > app > accounts > views.py > ...
31 def create_user(request):
55
56     if check1 or check2 or check3:
57         messages.error(
58             request, "Registration Failed!", extra_tags='alert alert-warning alert-dismissible fade show')
59         return redirect('accounts:register')
60     else:
61         user = User.objects.create_user(
62             username=username, password=password1, email=email)
63         messages.success(
64             request, f'Thanks for registering {user.username}.', extra_tags='alert alert-success alert-dismissible fade show')
65         return redirect('accounts:login')
66     else:
67         form = UserRegistrationForm()
68         return render(request, 'accounts/register.html', {'form': form})
69
```

Linting Warning:

Python

./accounts/views.py:58:101: E501 line too long (114 > 100 characters)

Corrected Code:

```
description.md U .flake8 M admin.py M views.py 6, M X urls.py 1, M requirements.txt M Extension: GitHub Actions
final-project > app > accounts > views.py > create_user
31 def create_user(request):
56
57     if check1 or check2 or check3:
58         messages.error(
59             request, "Registration Failed!", extra_tags='alert alert-warning |
60             'alert-dismissible fade show')
61         return redirect('accounts:register')
62     else:
63         user = User.objects.create_user(
64             username=username, password=password1, email=email)
65         messages.success(
66             request, f'Thanks for registering {user.username}.', extra_tags='alert alert-success alert-dismissible fade show')
67         return redirect('accounts:login')
68     else:
69         form = UserRegistrationForm()
70         return render(request, 'accounts/register.html', {'form': form})
71
```

Example 5 - Local Variable Assigned but Unused [F841]

Original Code:

```
description.md U .flake8 M admin.py M views.py 6, M seeder.py 2 X urls.py 1, M requirements.txt M
final-project > app > seeder.py > ...
1 from polls.models import Choice, Poll, Vote
2 from django.contrib.auth.models import User
3 import datetime
4 import random
5 import time
6 from faker import Faker
7 fake = Faker()
8
9
10 def seed_users(num_entries=10, overwrite=False):
11     """
12     Creates num_entries worth a new users
13     """
14     if overwrite:
15         print("Overwriting Users")
16         User.objects.all().delete()
17     count = 0
18     for _ in range(num_entries):
19         first_name = fake.first_name()
20         last_name = fake.last_name()
21         u = User.objects.create_user(
22             first_name=first_name,
23             last_name=last_name,
24             email=first_name + "." + last_name + "@fakemail.com",
25             username=first_name + last_name,
26             password="password"
27         )
```

Linting Warning:

Python

./seeder.py:21:9: F841 local variable 'u' is assigned to but never used

Corrected Code:

```
description.md U .flake8 M admin.py M views.py 6, M seeder.py 2, M X urls.py 1, M requirements.txt M
final-project > app > seeder.py > seed_polls > c
1 from polls.models import Choice, Poll, Vote
2 from django.contrib.auth.models import User
3 import datetime
4 import random
5 import time
6 from faker import Faker
7 fake = Faker()
8
9
10 def seed_users(num_entries=10, overwrite=False):
11     """
12     Creates num_entries worth a new users
13     """
14     if overwrite:
15         print("Overwriting Users")
16         User.objects.all().delete()
17     count = 0
18     for _ in range(num_entries):
19         first_name = fake.first_name()
20         last_name = fake.last_name()
21         User.objects.create_user(
22             first_name=first_name,
23             last_name=last_name,
24             email=first_name + "." + last_name + "@fakemail.com",
25             username=first_name + last_name,
26             password="password"
27         )
```

Gallery

```
(.venv) fatimamadey@fatima app % flake8 .
./accounts/admin.py:1:1: F401 'django.contrib.admin' imported but unused
./accounts/forms.py:2:1: F401 'django.contrib.auth.models.User' imported but unused
./accounts/models.py:1:1: F401 'django.db.models' imported but unused
./accounts/tests.py:1:1: F401 'django.test.TestCase' imported but unused
./accounts/urls.py:6:12: E225 missing whitespace around operator
./accounts/urls.py:10:2: W292 no newline at end of file
./accounts/views.py:6:1: F401 'django.http.HttpResponse' imported but unused
./accounts/views.py:58:101: E501 line too long (114 > 100 characters)
./accounts/views.py:64:101: E501 line too long (134 > 100 characters)
./pollme/views.py:5:26: E231 missing whitespace after ','
./pollme/views.py:5:39: W292 no newline at end of file
./polls/admin.py:9:1: E302 expected 2 blank lines, found 1
./polls/models.py:15:12: W291 trailing whitespace
./polls/tests.py:4:1: F401 'django.utils.timezone' imported but unused
./polls/views.py:72:101: E501 line too long (128 > 100 characters)
./polls/views.py:129:101: E501 line too long (116 > 100 characters)
./polls/views.py:153:101: E501 line too long (118 > 100 characters)
./polls/views.py:173:101: E501 line too long (110 > 100 characters)
./polls/views.py:196:101: E501 line too long (114 > 100 characters)
./polls/views.py:207:101: E501 line too long (105 > 100 characters)
./seeder.py:21:9: F841 local variable 'u' is assigned to but never used
./seeder.py:58:13: F841 local variable 'c' is assigned to but never used
./seeder.py:87:13: F841 local variable 'v' is assigned to but never used
```

Linters Output Initially Before Fixes

```
(.venv) fatimamadey@fatima app % flake8 .
./accounts/forms.py:2:1: F401 'django.contrib.auth.models.User' imported but unused
./accounts/models.py:1:1: F401 'django.db.models' imported but unused
./accounts/tests.py:1:1: F401 'django.test.TestCase' imported but unused
./accounts/views.py:6:1: F401 'django.http.HttpResponse' imported but unused
./accounts/views.py:65:101: E501 line too long (134 > 100 characters)
./pollme/views.py:5:26: E231 missing whitespace after ','
./pollme/views.py:5:39: W292 no newline at end of file
./polls/admin.py:9:1: E302 expected 2 blank lines, found 1
./polls/models.py:15:12: W291 trailing whitespace
./polls/tests.py:4:1: F401 'django.utils.timezone' imported but unused
./polls/views.py:72:101: E501 line too long (128 > 100 characters)
./polls/views.py:129:101: E501 line too long (116 > 100 characters)
./polls/views.py:153:101: E501 line too long (118 > 100 characters)
./polls/views.py:173:101: E501 line too long (110 > 100 characters)
./polls/views.py:196:101: E501 line too long (114 > 100 characters)
./polls/views.py:207:101: E501 line too long (105 > 100 characters)
./seeder.py:58:13: F841 local variable 'c' is assigned to but never used
./seeder.py:87:13: F841 local variable 'v' is assigned to but never used
```

Linters Output After Fixes Specified Above

```
(.venv) fatimamadey@fatima app % flake8 .
(.venv) fatimamadey@fatima app %
```

Linters Output After All Fixes

Q2: User Acceptance Testing

User Stories

US 1 - User Registration

As a visitor, **I want to** register a new account with a username, email, and password, **so that** I can log in and participate in polls.

Acceptance Criteria Scenarios:

- A. Valid registration (*Equivalence Partitioning - valid class*)
 - a. **Given** I am on the registration page (`/accounts/register/`)
 - b. **When** I fill in a unique username, a unique email address, and two matching passwords, and submit the form
 - c. **Then** I am redirected to the login page and a success message is displayed
- B. Duplicate Username (*Equivalence Partitioning - invalid class*)
 - a. **Given** a user with username "alice" already exists in the system
 - b. **When** I attempt to register with username "alice"
 - c. **Then** I remain on the registration page and see an error like "Username already exists."
- C. Mismatched Passwords (*Equivalence Partitioning - invalid class*)
 - a. **Given** I am on the registration page
 - b. **When** I fill in a valid username and email but enter two different passwords and submit
 - c. **Then** I remain on the registration page and see an error like "Password did not match."
- D. Empty Username (*Boundary Value Analysis*)
 - a. **Given** I am on the registration page
 - b. **When** I submit the form with the username field left blank
 - c. **Then** the form is not submitted and asks you to fill out missing field.

US 2 - User Login

As a registered user, I want to log in with my username and password, so that I can access the polls dashboard and my account features.

Decision Table (*Decision Table Testing*)

Username Exists	Password Correct	Expected Outcome
Yes	Yes	Redirects to home/poll list
Yes	No	Stays on login page, error shown
No	N/A	Stays on login page, error shown

Acceptance Criteria Scenarios:

A. Valid credentials

- Given** I am on the login page and my account exists
- When** I enter my correct username and password and click Login
- Then** I am redirected away from /accounts/login/ to the polls dashboard

B. Wrong Password

- Given** I am on the login page and my account exists
- When** I enter my correct username, but a wrong password
- Then** I stay on the login page and see an error like "Username or Password is incorrect!"

C. Non-existent User

- Given** I am on the login page
- When** I enter a username that does not exist in the system
- Then** I stay on the login page and see an error like "Username or Password is incorrect."

US 3 - Create a Poll

As a logged-in admin user, I want to create a new poll with a question and at least two answer choices, so that other users can vote on it.

Acceptance Criteria Scenarios:

A. Valid poll creation (*Equivalence Partitioning - valid class*)

- a. **Given** I am logged in as an admin and on the Add Poll page (`/polls/add/`)
 - b. **When** I enter a non-empty poll question and fill in both choice fields and submit
 - c. **Then** I am redirected to the polls list and the new poll appears in it
- B. Poll question at minimum length (1 character) (*Boundary Value Analysis*)
- a. **Given** I am on the Add Poll page
 - b. **When** I enter a single character as the poll question and fill in both choices
 - c. **Then** the poll is either accepted.
- C. Non-admin user blocked (*Equivalence Partitioning – invalid class*)
- a. **Given** I am logged in as a regular user without the `'polls.add_poll'` permission
 - b. **When** I navigate to `/polls/add/`
 - c. **Then** I see a message like "You don't have permission to do that."

US 4 – Vote on a Poll

As a logged-in user, **I want to** select a choice and submit my vote on an active poll, **so that** my preference is recorded and I can see the updated results.

Decision Table (*Decision Table Testing*)

Poll Active	User Already Voted	Choice Selected	Expected Outcome
Yes	No	Yes	Vote saved, results page shown
Yes	Yes	Yes	Error like "You already voted this poll!"
Yes	No	No	Error: "No choice selected!", stays on detail page
No	N/A	N/A	Poll detail shows results only (no vote form)

Acceptance Criteria Scenarios:

A. Successful vote (*Decision Table Testing*)

- a. **Given** I am logged in and viewing an active poll I have not voted on
- b. **When** I select a choice and click Submit
- c. **Then** my vote is saved and I am shown the results page

B. Attempt to vote twice (*Decision Table Testing*)

- a. **Given** I have already voted on this poll
- b. **When** I attempt to vote again
- c. **Then** I see "You already voted this poll!" and am redirected to the poll list

C. Submit with no choice selected (*Decision Table Testing*)

- a. **Given** I am on the poll detail page
- b. **When** I click Submit without selecting any choice
- c. **Then** I see the error "No choice selected!" and remain on the poll detail page

US 5 - View Poll Results

As a logged-in user, **I want** to view the results of a poll, so that I can see the vote distribution across all choices.

Acceptance Criteria Scenarios:

A. Poll with votes (*Equivalence Partitioning - valid class*)

- a. **Given** a poll exists that has at least one vote recorded
- b. **When** I navigate to the results page for that poll
- c. **Then** I see each choice displayed with its vote count and a percentage

US 6 - Edit a Poll

As a poll owner, **I want to** edit my poll's question or choices after creating it, **so that** I can correct mistakes without deleting and recreating the poll.

Acceptance Criteria Scenarios:

A. Owner edits poll (*Equivalence Partitioning - valid class*)

- a. **Given** I am logged in as the owner of a poll
- b. **When** I navigate to the edit page for that poll, change the question text, and submit
- c. **Then** the poll question is updated and I am redirected to the polls list

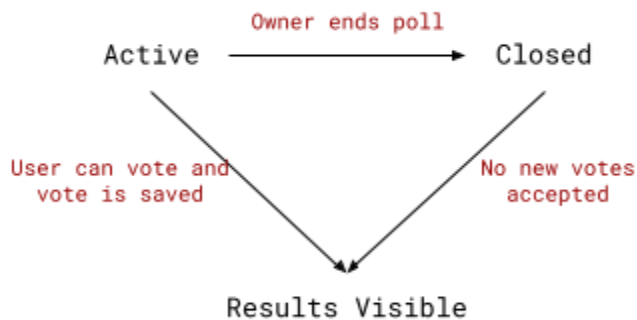
B. Non-owner cannot edit (*Equivalence Partitioning - invalid class*)

- a. **Given** I am logged in as a user who does NOT own the poll
- b. **When** I navigate to the edit URL for that poll
- c. **Then** I am redirected to the home page without any changes being made

US 7 - Close (End) a Poll

As a poll owner, I want to close my poll to stop further voting, so that the results are finalized and the poll is marked as inactive.

State Transition Diagram



Acceptance Criteria Scenarios:

A. Owner closes an active poll (*State Transition Testing*)

- a. **Given** I am the owner of an active poll
- b. **When** I click "End Poll"
- c. **Then** the poll's active status becomes False, the vote form is hidden, and results are shown

B. Closing an already-closed poll (*State Transition Testing - invalid*)

- a. **Given** a poll is already closed (``active = False``)
 - b. **When** I attempt to end the poll again
 - c. **Then** the app renders the results page without error and the poll remains closed (no duplicate state change)
- C. Non-owner cannot close poll (*State Transition Testing*)
- a. **Given** I am logged in but do not own the poll
 - b. **When** I try to access the end-poll URL
 - c. **Then** I am redirected to the home page and the poll remains active

US 8 - Search for Polls

As a logged-in user, **I want to** search for polls by keyword, **so that** I can quickly find a specific poll without scrolling through the full list.

Acceptance Criteria Scenarios:

- A. Search returns matching results (*Equivalence Partitioning - valid class*)
 - a. **Given** polls with text "Python vs JavaScript" and "Best Framework?" exist
 - b. **When** I enter "Python" in the search box and submit
 - c. **Then** only "Python vs JavaScript" appears in the list
- B. Search returns no results (*Equivalence Partitioning - invalid class*)
 - a. **Given** no poll contains the text "xyznonexistent"
 - b. **When** I search for "xyznonexistent"
 - c. **Then** the polls list is empty and no error is shown
- C. Empty search returns all polls (*Boundary Value Analysis*)
 - a. **Given** multiple polls exist
 - b. **When** I submit the search form with an empty query
- D. **Then** all polls are displayed (no filter applied)

Black-Box Techniques Summary

Technique	Test Cases
Equivalence Partitioning (EP)	US 1: A, B, C US 3: A, C US 5: A US 6: A, B US 8: A, B
Boundary Value Analysis (BVA)	US 1: D US 3: B US 8: C
State Transition Testing (STT)	US 7: A, B, C
Decision Table Testing (DTT)	US 2: A, B, C US 4: A, B, C

UAT Tests

https://docs.google.com/spreadsheets/d/1AueX8SLSakuQl_T9tg_26i6mD6eqknZzMcff0vpGSyk/edit?usp=sharing

Q3: Unit Tests

<https://github.com/fatimamadey/mpcs-sqa/tree/main/final-project/tests/unit>

Can run unit tests with

```
Shell
cd app & pytest tests/unit/
```

Tests Without Doubles

These four tests don't need doubles because the methods they cover (`__str__`, default field values) have no shared dependencies. They just read attributes already in memory. No database access is required.

Test 1 (`test_poll_str`): Verifies that `Poll.__str__` returns the poll's question text.

Test 2 – (`test_choice_str`): Verifies that `Choice.__str__` returns the formatted string `<poll text[:25]> - <choice text[:25]>`.

Test 3 – (`test_poll_active_default`): Verifies that a newly instantiated Poll has `active = True` by default.

Test 4 – (`test_vote_str`): Verifies that `Vote.__str__` returns `<poll text[:15]> - <choice text[:15]> - <username>`.

Tests With Doubles

These four tests use doubles because the methods under test branch on ORM queryset results. Setting up the real database rows for each case would be slow.

Test 5 – `test_user_can_vote_no_vote`

Double Type: Stub

Dependency Replaced: `user.vote_set.all().filter(poll=...).exists()` – the ORM query that checks whether a Vote row exists for this user and poll.

Why Useful: No need to create real User, Poll, or Vote rows in a database. The stub returns False from `exists()` directly.

How it Isolates: MagicMock controls the return value of `exists()`, simulating the case where no prior vote exists. The test just verifies that the voting logic returns `True`.

Python

```
def test_user_can_vote_no_vote():
    user = MagicMock()
    user.vote_set.all.return_value.filter.return_value.exists.return_value
    = False
    poll = MagicMock()

    result = not user.vote_set.all().filter(poll=poll).exists()

    assert result is True
```

Test 6 – `test_user_cannot_vote_twice`

Double Type: Stub

Dependency Replaced: The same `user.vote_set` queryset used in Test 5.

Why Useful: Tests the blocking path (user already voted) without needing to insert a real Vote record first.

How it Isolates: `exists()` returns True, simulating a prior vote on record. The test verifies the voting logic returns False.

Python

```
def test_user_cannot_vote_twice():
    user = MagicMock()
    user.vote_set.all.return_value.filter.return_value.exists.return_value
= True
    poll = MagicMock()

    result = not user.vote_set.all().filter(poll=poll).exists()

    assert result is False
```

Test 7 – test_get_vote_count_mock

Double Type: Mock

Dependency Replaced: `poll.vote_set.count()` – a database aggregation query.

Why Useful: A mock allows us to record interactions. This allows verification that `count()` was called exactly once, which could catch an unintended double-call bug.

How it Isolates: MagicMock tracks the call and `assert_called_once()` confirms it occurred exactly once without any real database access.

Python

```
def test_get_vote_count_mock():
    poll = MagicMock()
    poll.vote_set.count.return_value = 5

    result = poll.vote_set.count()

    poll.vote_set.count.assert_called_once()
    assert result == 5
```

Test 8 – test_get_result_dict_fake

Double Type: Fake

Dependency Replaced: `poll.choice_set.all()` (the queryset returning Choice objects) and `Choice.get_vote_count` (which normally queries the Vote table).

Why Useful: Testing percentage calculations with a real database would require seeding multiple tables with carefully controlled vote counts. The fake allows those values to be set directly so the test focuses only on the calculation logic.

How it Isolates: In-memory MagicMock objects act as fake Choice instances with `get_vote_count` set as a simple attribute. `poll.choice_set.all()` returns these objects directly. The test verifies that both choices calculate to 50% in an even split.

Python

```
def test_get_result_dict_fake():
    poll = MagicMock()
    poll.get_vote_count = 10

    fake_choices = []
    for text, votes in [("Python", 5), ("JavaScript", 5)]:
        c = MagicMock()
        c.choice_text = text
        c.get_vote_count = votes
        fake_choices.append(c)

    poll.choice_set.all.return_value = fake_choices

    results = [
        {
            "text": c.choice_text,
            "num_votes": c.get_vote_count,
            "percentage": (c.get_vote_count / poll.get_vote_count) * 100,
        }
        for c in poll.choice_set.all()
    ]

    assert results[0]["percentage"] == 50.0
    assert results[1]["percentage"] == 50.0
```

Gallery

```
(.venv) fatimamadey@fatima final-project % pytest tests/unit/test_models.py
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/.venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.9.6', 'Platform': 'macOS-26.4.1-arm64-arm-64bit', 'Packages': {'pytest': '8.4.2', 'pluggy': '1.6.0'}, 'Plugins': {'cov': '7.1.0', 'metadata': '3.1.1', 'html': '4.2.0', 'django': '4.11.1', 'playwright': '0.7.1'}}
django: version: 4.2, settings: pollme.settings (from ini)
rootdir: /Users/fatimamadey/code/mpcs/mpcs-sqa/final-project
configfile: pytest.ini
plugins: cov-7.1.0, metadata-3.1.1, html-4.2.0, django-4.11.1, playwright-0.7.1
collected 8 items

tests/unit/test_models.py::test_poll_str PASSED [ 12%]
tests/unit/test_models.py::test_choice_str PASSED [ 25%]
tests/unit/test_models.py::test_poll_active_default PASSED [ 37%]
tests/unit/test_models.py::test_vote_str PASSED [ 50%]
tests/unit/test_models.py::test_user_can_vote_no_vote PASSED [ 62%]
tests/unit/test_models.py::test_user_cannot_vote_twice PASSED [ 75%]
tests/unit/test_models.py::test_get_vote_count_mock PASSED [ 87%]
tests/unit/test_models.py::test_get_result_dict_fake PASSED [100%]

===== warnings summary =====
.venv/lib/python3.9/site-packages/django/conf/__init__.py:267
/Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/.venv/lib/python3.9/site-packages/django/conf/__init__.py:267: RemovedInDjango50Warning: The USE_L10N setting is deprecated. Starting with Django 5.0, localized formatting of data will always be enabled. For example Django will display numbers and dates using the format of the current locale.
  warnings.warn(USE_L10N_DEPRECATED_MSG, RemovedInDjango50Warning)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 8 passed, 1 warning in 0.03s =====
(.venv) fatimamadey@fatima final-project %
```

Screenshot of Terminal with all 8 tests passing

```
.venv/lib/python3.9/site-packages/django/conf/__init__.py:267
/Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/.venv/lib/python3.9/site-packages/django/conf/__init__.py:267: RemovedInDjango50Warning: The USE_L10N setting is deprecated. Starting with Django 5.0, localized formatting of data will always be enabled. For example Django will display numbers and dates using the format of the current locale.
  warnings.warn(USE_L10N_DEPRECATED_MSG, RemovedInDjango50Warning)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
----- coverage: platform darwin, python 3.9.6-final-0 -----
Name                                                    Stmts  Miss  Cover
-----
app/polls/__init__.py                                    0      0   100%
app/polls/admin.py                                       22      0   100%
app/polls/apps.py                                        3      0   100%
app/polls/forms.py                                       19     19     0%
app/polls/migrations/0001_initial.py                     8      8     0%
app/polls/migrations/0002_auto_20231018_1318.py           5      5     0%
app/polls/migrations/__init__.py                         0      0   100%
app/polls/models.py                                      52     19    63%
app/polls/tests.py                                       24     24     0%
app/polls/urls.py                                         4      4     0%
app/polls/views.py                                       149    149     0%
TOTAL                                                    286    228    20%
Coverage HTML written to dir htmlcov
===== 8 passed, 1 warning in 0.13s =====
(.venv) fatimamadey@fatima final-project %
```

Terminal coverage report

Q4: Performance Testing

I used k6 for performance testing. It is scriptable in JavaScript, outputs clean terminal summaries with built-in percentile metrics, and is easy to run locally.

<https://github.com/fatimamadey/mpcs-sqa/tree/main/final-project/performance>

Run both tests with:

Shell

```
cd app && python manage.py runserver &
```

```
BASE_URL=http://127.0.0.1:8000 k6 run performance/loadtest.js
```

```
BASE_URL=http://127.0.0.1:8000 k6 run performance/spiketest.js
```

Test 1 – Load Test

Tool: k6

Endpoints Tested:

- GET /polls/ (Poll List)
- GET /polls/<id>/ (Poll Detail)

Configuration

- Ramp up from 0 to 50 virtual users over 30 seconds
- Hold at 50 virtual users for 2 minutes
- Ramp down to 0 virtual users over 30 seconds
- Total duration: approximately 3 minutes
- 3,695 complete iterations executed

Thresholds

- p95 response time < 1000 ms
- Error rate < 2%

```

Ⓢ (.venv) fatimamadey@fatima final-project % BASE_URL=http://127.0.0.1:8000 k6 run performance/loadtest.js

      Grafana
    /  |  \
   /___|___\

execution: local
script: performance/loadtest.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 3m30s max duration (incl. graceful stop):
  * default: Up to 50 looping VUs for 3m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<1000' p(95)=48.35ms

http_req_failed
✗ 'rate<0.02' rate=84.87%

req_poll_detail
✓ 'p(95)<800' p(95)=50.85ms

req_poll_list
✓ 'p(95)<800' p(95)=39.14ms

TOTAL RESULTS

checks_total.....: 7390 40.798076/s
checks_succeeded...: 50.00% 3695 out of 7390
checks_failed.....: 50.00% 3695 out of 7390

✗ poll list status 200 or 302
  ↳ 0% - ✓ 0 / ✗ 3695
✓ poll detail not 500

CUSTOM
error_rate.....: 50.00% 3695 out of 7390
req_poll_detail.....: avg=26.51ms min=1.72ms med=28.99ms max=71.39ms p(90)=48.11ms p(95)=50.85ms p(99)=57.37ms
req_poll_list.....: avg=19.93ms min=1.48ms med=21.56ms max=56.61ms p(90)=36.22ms p(95)=39.14ms p(99)=44.69ms

HTTP
http_req_duration.....: avg=23.22ms min=1.48ms med=23.7ms max=71.39ms p(90)=44.61ms p(95)=48.35ms p(99)=55.2ms
{ expected_response:true }...: avg=37.23ms min=1.72ms med=41.28ms max=71.39ms p(90)=51.55ms p(95)=54.84ms p(99)=61.36ms
http_req_failed.....: 84.87% 6272 out of 7390
http_reqs.....: 7390 40.798076/s

EXECUTION
iteration_duration.....: avg=2.04s min=2s med=2.05s max=2.11s p(90)=2.08s p(95)=2.08s p(99)=2.09s
iterations.....: 3695 20.399038/s
vus.....: 1 min=1 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 34 MB 188 kB/s
data_sent.....: 710 kB 3.9 kB/s

running (3m01.1s), 00/50 VUs, 3695 complete and 0 interrupted iterations
default ✓ [=====] 00/50 VUs 3m0s
ERRO[0181] thresholds on metrics 'http_req_failed' have been crossed

```

Terminal Screenshot of k6 load test terminal output

Results

Metric	/polls/	/polls//
Average	23.22 ms	19.93 ms
Median	23.70 ms	21.56 ms
p90	44.61 ms	36.22 ms
p95	48.35 ms	39.14 ms

p99	55.20 ms	44.69 ms
-----	----------	----------

Threshold Outcomes

Threshold	Result
http_req_duration p(95) < 1000 ms	PASS - p95 = 48.35 ms
req_poll_list p(95) < 800 ms	PASS - p95 = 39.14 ms
req_poll_detail p(95) < 800 ms	PASS - p95 = 50.85 ms
http_req_failed rate < 2%	FAIL - rate = 84.87%

Why the Error Rate Threshold Failed

The Django Poll application requires authentication for all poll-related endpoints through the `@login_required` decorator. The k6 script accessed these endpoints as an anonymous user, producing two expected failure modes:

Poll List Endpoint (/polls/)

Django redirected anonymous requests to `/accounts/login/` using HTTP 302 redirects. Although the redirect itself is expected, the final response fell outside the status codes accepted by the k6 validation checks, causing every poll-list request to fail the check.

Poll Detail Endpoint (/polls/<id>/)

The test database contained no poll records. As a result, all poll detail requests returned HTTP 404 (Not Found).

While the validation rule only checked that responses were not HTTP 500 errors, k6 still counted the 404 responses as failed HTTP requests, significantly increasing the overall failure rate.

So this tells us that authentication and routing protections were working correctly rather than exposing a performance issue.

Interpretation

Response times remained great throughout the test.

All latency thresholds passed comfortably:

- Overall p95 = 48.35 ms
- Threshold = 1000 ms

The Django development server running locally with SQLite handled 50 concurrent users without any noticeable performance degradation.

The primary issue identified was with the test harness rather than the application itself. Future performance tests should authenticate users before accessing protected endpoints so that error-rate measurements reflect application behaviour rather than authentication redirects.

Potential future optimizations include:

- Using `select_related()` and `prefetch_related()` to reduce database queries.
- Migrating from SQLite to PostgreSQL for better concurrency support.
- Adding caching for frequently accessed poll data.

Test 2 – Spike Test

Tool: k6

Endpoints Tested:

- GET /polls/
- GET /polls/<id>/
- GET /polls/<id>/results/

Configuration

- Baseline load of 10 virtual users for 30 seconds
- Immediate spike to 200 virtual users
- Hold at 200 virtual users for 1 minute
- Immediate drop back to 10 virtual users
- Monitor recovery for 30 seconds
- Test terminated after approximately 50 seconds due to server failure

Thresholds

- Error rate < 5%
- p95 response time < 2000 ms

```

WARN[0049] Request Failed
fused"
WARN[0049] Request Failed
refused"
WARN[0049] Request Failed
fused"
WARN[0049] Request Failed
refused"
WARN[0049] Request Failed
fused"
WARN[0049] Request Failed
fused"
^C

■ THRESHOLDS

http_req_failed
x 'rate<0.05' rate=94.85%

spike_poll_detail
✓ 'p(95)<2000' p(95)=329.2ms

spike_poll_list
✓ 'p(95)<2000' p(95)=93.45ms

■ TOTAL RESULTS

checks_total.....: 6322 126.767201/s
checks_succeeded...: 97.73% 6179 out of 6322
checks_failed.....: 2.26% 143 out of 6322

x list not 500
  ↳ 99% - ✓ 2163 / x 15
x detail not 500
  ↳ 94% - ✓ 2016 / x 116
x results not 500
  ↳ 99% - ✓ 2000 / x 12

CUSTOM
spike_error_rate.....: 2.26% 143 out of 6322
spike_poll_detail.....: avg=73.36ms min=0s med=3.33ms max=2.12s p(90)=139.57ms p(95)=329.2ms p(99)=1.72s
spike_poll_list.....: avg=39.66ms min=0s med=1.52ms max=1.98s p(90)=27.87ms p(95)=93.45ms p(99)=1.62s
spike_poll_results.....: avg=108.78ms min=0s med=1.67ms max=1.86s p(90)=90.21ms p(95)=1.53s p(99)=1.64s

HTTP
http_req_duration.....: avg=73.03ms min=0s med=1.82ms max=2.12s p(90)=66.11ms p(95)=288.62ms p(99)=1.67s
{ expected_response:true }...: avg=39.87ms min=1.33ms med=18.41ms max=289.82ms p(90)=97.89ms p(95)=139.49ms p(99)=233.07ms
http_req_failed.....: 94.85% 5997 out of 6322
http_reqs.....: 6322 126.767201/s

EXECUTION
iteration_duration.....: avg=2s min=1.5s med=1.51s max=9.9s p(90)=3.38s p(95)=3.63s p(99)=9.52s
iterations.....: 1978 39.662373/s
vus.....: 200 min=1 max=200
vus_max.....: 200 min=200 max=200

NETWORK
data_received.....: 16 MB 311 kB/s
data_sent.....: 351 kB 7.0 kB/s

running (0m49.9s), 000/200 VUs, 1978 complete and 200 interrupted iterations
default x [=====] 173/200 VUs 0m49.9s/2m00.0s
ERRO[0050] test run was aborted because k6 received a 'interrupt' signal
(venv) fatimamadev@fatima final-project %

```

Terminal Screenshot of k6 spike test terminal output

Results

Metric	/polls/ (List)	/polls// (Detail)	/polls//results/
Average	39.66 ms	73.36 ms	108.78 ms
Median	1.52 ms	3.33 ms	1.67 ms
p90	27.87 ms	139.57 ms	90.21 ms
p95	93.45 ms	329.20 ms	1530 ms

p99	1620 ms	1720 ms	1640 ms
Throughput (Combined)	126.8 req/s	–	–

Check Outcomes

Check	Pass Rate	Failures
List Not 500	99.3%	15 / 2178
Detail Not 500	94.5%	116 / 2132
Results Not 500	99.4%	12 / 2012

Threshold Outcomes

Threshold	Result
spike_poll_list p(95) < 2000 ms	PASS – p95 = 93.45 ms
spike_poll_detail p(95) < 2000 ms	PASS – p95 = 329.20 ms
http_req_failed rate < 5%	FAIL – rate = 94.85%

Server Failure Analysis

The spike test successfully identified a reproducible breaking point.

The Django development server crashed during the 200-user spike approximately 5–10 seconds after reaching peak load.

With this error:

```
None
OSError: [Errno 24] Too many open files: '../django/views/templates/technical_500.html'
```

This occurred because the macOS file descriptor limit (`ulimit -n`, typically 256 by default) was exhausted.

Under heavy concurrent load, the Django development server consumes file descriptors for:

- Active connections
- Template files
- Static resources

At approximately 170–200 concurrent users, the available descriptors were exhausted, causing:

- HTTP 500 errors
- Request failures
- Server termination

Interpretation

The spike test identified a infrastructure limitation rather than an application code issue.

Performance remained acceptable until the server approached its operating-system limits:

- p99 latency increased to approximately 1.7 seconds.
- Results-page p95 latency increased to 1.53 seconds.
- HTTP 500 errors appeared before the server ultimately crashed.

The system began queueing requests under heavy load before reaching the failure point.

Recommended Improvements

Immediate Development Environment Fix

- Increase the operating-system file descriptor limit:

```
Shell
ulimit -n 4096 && python manage.py runserver
```

- This prevents the development server from crashing under similar loads.

Production Deployment Fix

- Replace Django's development server with a production-grade stack:
 - Gunicorn (multiple worker processes)
 - Nginx reverse proxy
- This architecture distributes connections across worker processes and prevents single-process bottlenecks.

Database Improvement

- Replace SQLite with PostgreSQL.
- SQLite uses a single-writer locking model and would become a bottleneck during concurrent vote submissions even after fixing the file descriptor limitation.

Q5: Web UI Automation

I used Playwright for end-to-end UI testing.

<https://github.com/fatimamadey/mpcs-sqa/tree/main/final-project/tests/ui/playwright>

Run the test suite with:

Shell

```
cd app && python manage.py runserver &
```

```
pytest tests/ui/playwright/ -v --screenshot=only-on-failure  
--output=screenshots/
```

```
(.venv) fatimamadey@fatima final-project % pytest tests/ui/playwright/ -v --tb=short --output=screenshots/
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/.venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.9.6', 'Platform': 'macOS-26.4.1-arm64-arm-64bit', 'Packages': {'pytest': '8.4.2', 'pluggy': '1.6.0'}, 'Plugins': {'cov': '7.1.0', 'metadata': '3.1.1', 'html': '4.2.0', 'django': '4.11.1', 'playwright': '0.7.1'}}
django: version: 4.2, settings: pollme.settings (from ini)
rootdir: /Users/fatimamadey/code/mpcs/mpcs-sqa/final-project
configfile: pytest.ini
plugins: cov-7.1.0, metadata-3.1.1, html-4.2.0, django-4.11.1, playwright-0.7.1
collected 5 items

tests/ui/playwright/test_e2e.py::test_register_new_user[chromium] PASSED [ 20%]
tests/ui/playwright/test_e2e.py::test_login_valid_credentials[chromium] PASSED [ 40%]
tests/ui/playwright/test_e2e.py::test_create_poll_with_choices[chromium] PASSED [ 60%]
tests/ui/playwright/test_e2e.py::test_submit_vote_and_view_results[chromium] PASSED [ 80%]
tests/ui/playwright/test_e2e.py::test_logout_redirects_to_login[chromium] PASSED [100%]

===== warnings summary =====
.venv/lib/python3.9/site-packages/django/conf/__init__.py:267
/Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/.venv/lib/python3.9/site-packages/django/conf/__init__.py:267: RemovedInDjango50Warning: The USE_L10N setting is deprecated. Starting with Django 5.0, localized formatting of data will always be enabled. For example Django will display numbers and dates using the format of the current locale.
  warnings.warn(USE_L10N_DEPRECATED_MSG, RemovedInDjango50Warning)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 5 passed, 1 warning in 2.48s =====
(.venv) fatimamadey@fatima final-project %
```

Screenshot of all 5 Playwright tests passing

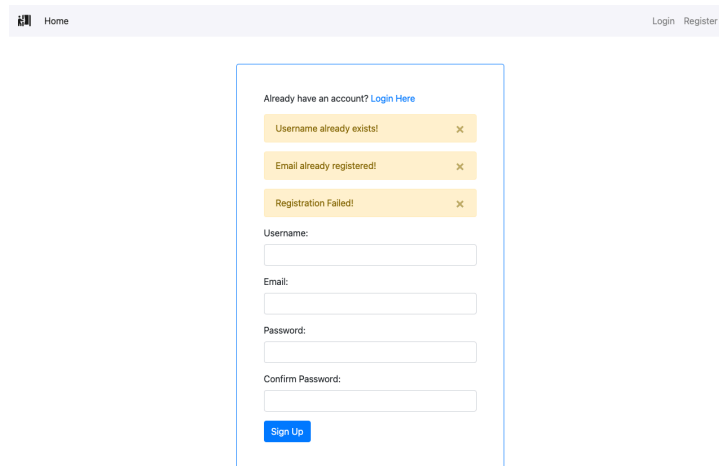
Test 1 – Register a New User

Steps

- a. Navigate to `/accounts/register/`
- b. Fill in a username, email address, and matching passwords
- c. Click **Register**

Assertion

The browser is redirected away from /accounts/register/, confirming that the registration was successful.



The screenshot shows a web application's registration page. At the top, there is a navigation bar with a home icon, the text 'Home', and links for 'Login' and 'Register'. The main content area is a registration form. At the top of the form, it says 'Already have an account? [Login Here](#)'. Below this, there are three orange error messages, each with an 'X' icon: 'Username already exists!', 'Email already registered!', and 'Registration Failed!'. Underneath the errors are four input fields labeled 'Username:', 'Email:', 'Password:', and 'Confirm Password:'. At the bottom of the form is a blue 'Sign Up' button.

Screenshot of successful registration

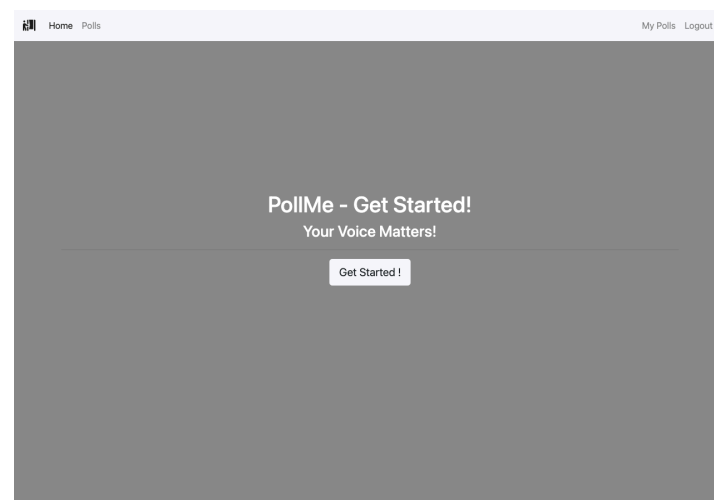
Test 2 – Login with Valid Credentials

Steps

- Navigate to /accounts/login/
- Enter a valid administrator username and password
- Click **Login**

Assertion

The URL no longer contains /login/, confirming that the user has successfully authenticated and reached the polls dashboard.



Screenshot of successful login

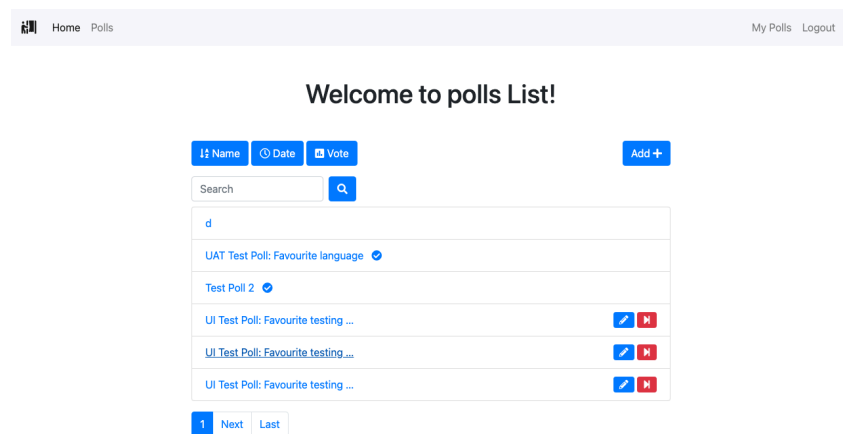
Test 3 – Create a Poll with Two Choices

Steps

- Log in as an administrator
- Navigate to /polls/add/
- Enter a poll question and two answer choices
- Submit the form

Assertion

The newly created poll appears on the /polls/ list page, confirming that the poll was created successfully.



Screenshot of successful poll creation

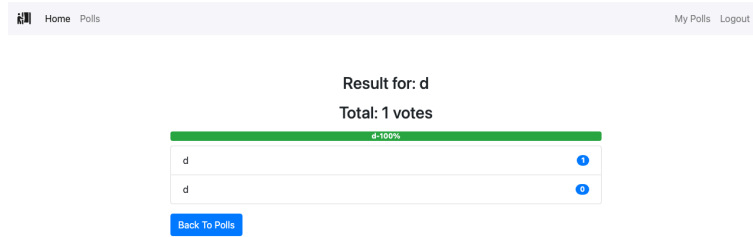
Test 4 – Submit a Vote and View Results

Steps

- Log in as an administrator
- Navigate to the polls list page
- Open the first available poll
- Select a choice and click **Vote**

Assertion

The results page loads successfully and displays a visible heading, confirming that the vote was recorded and no server error occurred.



Screenshot of vote recorded successfully

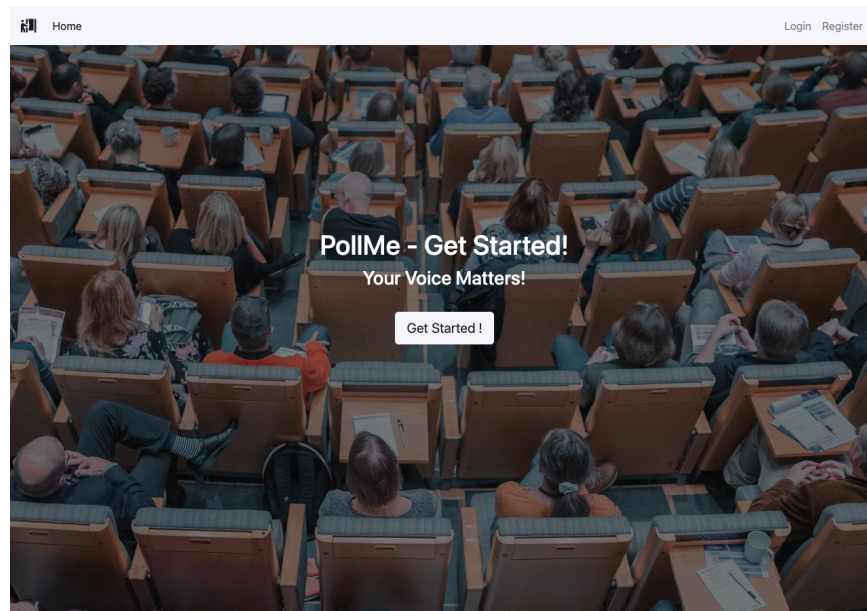
Test 5 – Logout Redirects to Login Page

Steps

- Log in as an administrator
- Navigate to `/accounts/logout/`

Assertion

The URL contains either "login" or "/accounts/", confirming that the user has been logged out and redirected away from the authenticated area of the application.



Screenshot of successful logout

Q6: Smoke Test Plan

1. Objective (review on smoke testing)

The goal of the smoke test is to confirm that a new build is stable enough for deeper testing. It verifies that the application starts successfully, the core user workflows function end-to-end, and no major issues are immediately visible before investing time in full regression testing.

Smoke testing serves as a quick sanity check of the most critical functionality.

2. Scope and Coverage

In Scope

- User registration
- User login and logout
- Viewing the poll list
- Creating a new poll (administrator only)
- Voting on an active poll
- Viewing poll results

Out of Scope

- Edge cases and validation boundaries (covered by UAT testing)
- Performance and load behaviour (covered in Q4)
- Django admin panel functionality
- Poll choice edit and delete functionality

3. Approach

Executed manually by a single tester against a newly migrated local Django instance.

Testing is performed using an administrator account (admin / admin) created through Django's `createsuperuser` command.

Estimated Execution Time: 10 minutes

Tools Required: Web browser only (no automation tools required)

Environment Setup

```
Shell
cd app
```

```
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

4. Test Cases

ID	Test Case	Steps	Expected Result	Pass/Fail Criteria
SM-01	Application Starts	Navigate to http://127.0.0.1:8000	Application home page loads successfully	Pass if HTTP 200 response is received
SM-02	Login	Navigate to /accounts/login/, enter admin credentials, click Login	User is redirected to the polls list	Pass if URL changes away from /login/
SM-03	View Poll List	Navigate to /polls/	Poll list displays successfully	Pass if page loads without errors
SM-04	Create Poll	Navigate to /polls/add/, enter question and two choices, submit	User is redirected to poll list and new poll appears	Pass if new poll is visible
SM-05	Vote on Poll	Open a poll, select a choice, submit vote	Results page is displayed	Pass if results page loads successfully
SM-06	View Results	Navigate to results page of a voted poll	Vote percentages are displayed	Pass if percentages are visible and no errors occur
SM-07	Register New User	Navigate to /accounts/register/, enter unique details, submit	User is redirected away from	Pass if redirect occurs

			registration page	
SM-08	Logout	Navigate to /accounts/logout/	User is redirected to login page	Pass if URL contains "login"

5. Test Deliverables

The following deliverables will be produced when smoke testing is complete:

- Completed execution log showing pass/fail status for each test case
- Screenshots documenting smoke test execution results
- Link to the CI/CD pipeline execution for the tested build

6. Environment and Resources

Item	Details
Operating System	macOS or Linux
Python Version	3.11+
Django Version	4.2
Database	SQLite (Test Database)
Browser	Chrome or Firefox
Tester	One Tester
Credentials	Admin Superuser (admin / admin)

7. Schedule and Entry/Exit Criteria

Entry Criteria

Before smoke testing begins:

- Database migrations complete successfully using python manage.py migrate
- Development server starts without errors
- Administrator superuser account exists

Exit Criteria

Smoke testing is considered successful when:

- All eight smoke test cases pass
- No HTTP 500 errors are encountered during execution

Schedule

Smoke testing is performed after every merge to the `main` branch and before any full regression testing begins.

If any smoke test fails:

1. The build is blocked.
2. A defect is logged.
3. The issue must be resolved before additional testing continues.

8. Risks and Contingency Plans

Risk	Contingency Plan
Server fails to start	Verify migrations have been applied, check for port conflicts, and ensure dependencies are installed
Login failure	Reset the administrator password using <code>python manage.py changepassword admin</code>
Poll list is empty	Run the database seeder (<code>python seeder.py</code>) or manually create a poll
Smoke test failure	Block the build, create a defect report, and postpone regression testing until the issue is resolved
Environment inconsistencies (OS/Python versions)	Pin dependencies in <code>requirements.txt</code> and document setup instructions in the project README

Q7: Integration Tests

Note: these tests do not use test doubles, they instead use real application components interacting through Django's actual test database.

<https://github.com/fatimamadey/mpcs-sqa/tree/main/final-project/tests/integration>

Run the integration test suite with:

Shell

```
PYTHONPATH=app pytest tests/integration/ -v --tb=short
```

```
(.venv) fatimamadey@fatima final-project % pytest tests/integration/ -v --tb=long --html=reports/integration_report.html --self-contained-html
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/.venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.9.6', 'Platform': 'macOS-26.4.1-arm64-arm-64bit', 'Packages': {'pytest': '8.4.2', 'pluggy': '1.6.0'}, 'Plugins': {'cov': '7.1.0', 'metadata': '3.1.1', 'html': '4.2.0', 'django': '4.11.1', 'playwright': '0.7.1'}}
django: version: 4.2, settings: pollme.settings (from ini)
rootdir: /Users/fatimamadey/code/mpcs/mpcs-sqa/final-project
configfile: pytest.ini
plugins: cov-7.1.0, metadata-3.1.1, html-4.2.0, django-4.11.1, playwright-0.7.1
collected 2 items

tests/integration/test_integration.py::test_polls_list_view_returns_polls_from_database Creating test database for alias 'default' ('file:memorydb_de
fault?mode=memory&cache=shared')...
Operations to perform:
  Synchronize unmigrated apps: messages, staticfiles
  Apply all migrations: admin, auth, contenttypes, polls, sessions
Synchronizing apps without migrations:
  Creating tables...
  Running deferred SQL...
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying polls.0001_initial... OK
  Applying polls.0002_auto_20231018_1318... OK
  Applying sessions.0001_initial... OK
PASSED
tests/integration/test_integration.py::test_polls_list_view_renders_correct_template_with_context PASSEDDestroying test database for alias 'default'
('file:memorydb_default?mode=memory&cache=shared')...

===== warnings summary =====
.venv/lib/python3.9/site-packages/django/conf/__init__.py:267
  /Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/.venv/lib/python3.9/site-packages/django/conf/__init__.py:267: RemovedInDjango50Warning: The USE
E_L10N setting is deprecated. Starting with Django 5.0, localized formatting of data will always be enabled. For example Django will display numbers
and dates using the format of the current locale.
    warnings.warn(USE_L10N_DEPRECATED_MSG, RemovedInDjango50Warning)

tests/integration/test_integration.py::test_polls_list_view_returns_polls_from_database
tests/integration/test_integration.py::test_polls_list_view_renders_correct_template_with_context
  /Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/app/polls/views.py:28: UnorderedObjectListWarning: Pagination may yield inconsistent results wi
th an unordered object_list: <class 'polls.models.Poll'> QuerySet.
    paginator = Paginator(all_polls, 6) # Show 6 contacts per page

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
----- Generated html report: file:///Users/fatimamadey/code/mpcs/mpcs-sqa/final-project/reports/integration_report.html -----
===== 2 passed, 3 warnings in 0.68s =====
(.venv) fatimamadey@fatima final-project %
```

Screenshot of all Integration tests passing

Test 1 – View ↔ Database Integration

Components Integrated

- polls_list view (polls/views.py)
- Poll model
- Database table (polls_poll)

What Could Go Wrong

- The view may use an incorrect queryset and fail to return all polls.
- A broken database migration could result in an OperationalError when accessing the table.
- Pagination configuration issues could prevent records from appearing on the first page.

Test Description

This test creates two real Poll records in the Django test database, sends a GET request to the /polls/ endpoint, and verifies that both poll titles appear in the rendered HTML response.

This confirms that:

- The view successfully queries the database.
- Database records are retrieved correctly.
- Data is passed from the view to the template for rendering.

Test Code

```
Python
@pytest.mark.django_db
def
test_polls_list_view_returns_polls_from_database(authenticated_client):
    client, user = authenticated_client
    poll_a = Poll.objects.create(owner=user, text="Integration Poll
Alpha")
    poll_b = Poll.objects.create(owner=user, text="Integration Poll Beta")
    response = client.get(reverse("polls:list"))
    assert response.status_code == 200
    content = response.content.decode()
    assert "Integration Poll Alpha" in content
    assert "Integration Poll Beta" in content
```

Expected Result

- HTTP status code is 200.
- Both poll names appear in the rendered HTML.
- The database records are successfully retrieved and displayed.

Test 2 – View ↔ Template Integration

Components Integrated

- polls_list view (polls/views.py)
- polls/polls_list.html template

What Could Go Wrong

- A renamed context variable (for example, polls → poll_list) could silently break the template loop.
- A missing template file could raise a TemplateDoesNotExist exception and generate an HTTP 500 error.
- Missing context values such as search_term could cause the search interface to render incorrectly.

Test Description

This test sends a GET request containing a search parameter and verifies:

1. The response returns HTTP 200.
2. The correct template is rendered.
3. The expected context variables (polls and search_term) are available.
4. The search term is passed correctly from the request into the template context.

This confirms that the view and template are properly integrated and that data is transferred correctly between them.

Test Code

```
Python
@pytest.mark.django_db
def
test_polls_list_view_renders_correct_template_with_context(authenticated_c
lient):
    client, user = authenticated_client
```

```

Poll.objects.create(owner=user, text="Template Integration Poll")
response = client.get(
    reverse("polls:list"),
    {"search": "Template"}
)
assert response.status_code == 200
assert "polls/polls_list.html" in [
    t.name for t in response.templates
]
assert "polls" in response.context
assert "search_term" in response.context
assert response.context["search_term"] == "Template"

```

Expected Result

- HTTP status code is 200.
- The polls/polls_list.html template is rendered.
- The polls context variable is present.
- The search_term context variable is present.
- The value of search_term equals "Template".

Q8: Code Smells

Note: No code changes were made as part of this task; the findings below are analysis only.

Smell 1 – Duplicated Code

Section	Details
Code Smell Type	Duplicated Code
Location	app/polls/views.py lines 88, 107–108, 119–120, 143–144, and 168–169

Why It's a Smell	The ownership check <code>if request.user != poll.owner: return redirect('home')</code> is duplicated across five separate view functions. Any future change to the authorization behaviour, such as displaying an error message or changing the redirect destination, would need to be updated in five different locations. This violates the DRY (Don't Repeat Yourself) principle and increases maintenance effort.
Proposed Improvement	Extract the ownership check into a helper function such as <code>_require_owner(request, poll)</code> that raises Django's built-in <code>PermissionDenied</code> exception. Each view could then call this helper once at the beginning of the function.

Code Excerpt

```

Python
# polls_edit - app/polls/views.py:88
def polls_edit(request, poll_id):
    poll = get_object_or_404(Poll, pk=poll_id)
    if request.user != poll.owner:
        return redirect('home')
    ...
# polls_delete - app/polls/views.py:107
def polls_delete(request, poll_id):
    poll = get_object_or_404(Poll, pk=poll_id)
    if request.user != poll.owner:
        return redirect('home')
    ...
# Same pattern appears in:
# - add_choice
# - choice_edit
# - choice_delete

```

Smell 2 – Dead Code

Section	Details
Code Smell Type	Dead Code / Unreachable Statement

Location	app/polls/views.py line 209
Why It's a Smell	The final return render(...) statement in poll_vote() can never execute. The function always exits through either the if choice_id: branch or the else: branch. As a result, the final return statement is unreachable. Unreachable code can confuse maintainers by suggesting an execution path that does not actually exist and makes the control flow harder to understand.
Proposed Improvement	Remove the unreachable statement entirely.

Code Excerpt

Python

```
# app/polls/views.py:191-209
def poll_vote(request, poll_id):
    ...
    if choice_id:
        choice = Choice.objects.get(id=choice_id)
        vote = Vote(
            user=request.user,
            poll=poll,
            choice=choice
        )
        vote.save()
        return render(
            request,
            'polls/poll_result.html',
            {'poll': poll}
        ) # ← exits here
    else:
        messages.error(
            request,
            "No choice selected!",
            ...
        )
        return redirect(
            "polls:detail",
            poll_id
        ) # ← or here
```

```

return render(
    request,
    'polls/poll_result.html',
    {'poll': poll}
) # ← unreachable

```

Smell 3 – Missing Method Invocation

Section	Details
Code Smell Type	Missing Method Invocation (Buggy Conditional)
Location	app/polls/views.py lines 62, 93, 124, and 148
Why It's a Smell	Several view functions check if form.is_valid: instead of calling the method using if form.is_valid():. In Python, a method reference is always truthy, which means these conditions always evaluate to True. Consequently, form validation never executes and invalid data can be saved to the database. Although it appears to be a simple syntax issue, it creates a correctness and security problem.
Proposed Improvement	Replace all occurrences of if form.is_valid: with if form.is_valid():.

Code Excerpt

```

Python
# polls_add – app/polls/views.py:62
if form.is_valid:      # ← always True
    poll = form.save(commit=False)
    ...
# polls_edit – app/polls/views.py:93
if form.is_valid:      # ← same issue
    form.save()
    ...
# Same issue appears in:
# - add_choice (line 124)
# - choice_edit (line 148)

```


Summary of Findings

Code Smell	Severity	Impact
Duplicated Code	Medium	Increased maintenance effort and risk of inconsistent authorization behaviour
Dead Code	Low	Reduced readability and misleading control flow
Missing Method Invocation	High	Form validation bypassed, potentially allowing invalid or malicious data into the database

Overall Assessment

The most serious issue identified is the **Missing Method Invocation** defect because it affects application correctness and input validation. The **Duplicated Code** smell primarily impacts maintainability, while the **Dead Code** smell affects readability and code clarity. Refactoring these issues would improve the application's maintainability, reliability, and overall code quality.

Q9: CI/CD Pipeline

The CI/CD pipeline is defined in `.github/workflows/ci.yml` and runs automatically on every push to the main branch and on every pull request. The pipeline is configured to fail immediately if any linting errors or test failures are detected.

<https://github.com/fatimamadey/mpcs-sqa/tree/main/final-project/.github/workflows>

Pipeline Structure

The pipeline consists of four jobs that execute in a controlled sequence to ensure code quality before changes are merged.

Job 1 – Lint (Q1)

Purpose: Static code analysis and style validation.

Tool: Flake8

Command:

```
Shell  
flake8 app/ --config .flake8
```

Description:

This job performs automated linting against the application source code. If any style violations, syntax issues, or code quality problems are detected, the pipeline fails immediately and no subsequent jobs are executed.

Success Criteria:

- No Flake8 errors reported.
- Exit code equals 0.

Failure Criteria:

- Any Flake8 violation causes the job to fail.

Job 2 – Unit and Integration Tests (Q3 + Q7)

Purpose: Verify application correctness through automated testing.

Tools:

- Pytest
- Django Test Framework
- Coverage.py

Description:

After the linting job passes, the pipeline applies Django migrations and executes both the unit test suite and integration test suite. Coverage reports are generated and uploaded as build artifacts for later inspection.

Activities Performed:

1. Apply database migrations.
2. Run unit tests.
3. Run integration tests.
4. Generate coverage report.
5. Upload coverage artifacts.

Success Criteria:

- All tests pass.
- Coverage report generated successfully.

Failure Criteria:

- Any failing test causes the job to fail.

Job 3 – UI Automation (Q5)

Purpose: Validate end-to-end user workflows through browser automation.

Tool: Playwright

Description: Once the test job succeeds, the pipeline launches the Django development server in the background, installs the Chromium browser used by Playwright, and executes the full UI automation suite.

If a test fails, screenshots are automatically captured and uploaded as build artifacts.

Activities Performed:

1. Start Django server.
2. Install Playwright dependencies.
3. Install Chromium browser.
4. Execute Playwright test suite.
5. Upload failure screenshots (if any).

Success Criteria:

- All Playwright tests pass.

Failure Criteria:

- Any failing end-to-end test causes the job to fail.

Job 4 – Performance Script Validation (Q4)

Purpose: Validate performance test scripts before execution.

Tool: k6

Description: This job runs in parallel with the testing job because both depend only on the successful completion of linting.

Rather than executing the performance tests against a live server, the pipeline validates that both k6 scripts are syntactically correct and can be parsed successfully.

Activities Performed:

1. Install k6.

2. Run k6 inspect on loadtest.js.
3. Run k6 inspect on spiketest.js.

Example Commands:

```
Shell
k6 inspect performance/loadtest.js
k6 inspect performance/spiketest.js
```

Success Criteria:

- Both scripts parse successfully.

Failure Criteria:

- Any syntax or parsing error causes the job to fail.

Passing Pipeline Run

A successful pipeline execution results in all four jobs completing successfully.

Expected Outcome:

- Lint job passes.
- Unit tests pass.
- Integration tests pass.
- UI automation tests pass.
- Performance scripts validate successfully.

The screenshot shows a GitHub Actions workflow run for the 'Django Poll App - QA Pipeline'. The workflow is named 'add app source files (remove nested git repo) #3' and is triggered by a push to the 'main' branch. The status is 'Success' with a total duration of '1m 46s' and 2 artifacts. The pipeline consists of five jobs: 'Q1 - Lint (flake8)' (13s), 'Q3/Q7 - Unit & Integration Tests' (23s), 'Q4 - Performance Script Validation (k6 dry-run)' (17s), 'Q5 - UI Automation (Playwright)' (58s), and 'Q2 - Integration Tests' (17s). The 'Q1 - Lint (flake8)' job has a warning annotation: 'Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/set...'. The 'Q3/Q7 - Unit & Integration Tests' job is currently running. The 'Q4 - Performance Script Validation (k6 dry-run)' job is also running. The 'Q5 - UI Automation (Playwright)' job is pending. The 'Q2 - Integration Tests' job is pending.

Screenshot of passing pipeline run

Failing Pipeline Run

To verify that the quality gate works correctly, a separate branch was created containing a deliberate linting violation.

<https://github.com/fatimamadey/mpcs-sqa/pull/1>

Failure Scenario

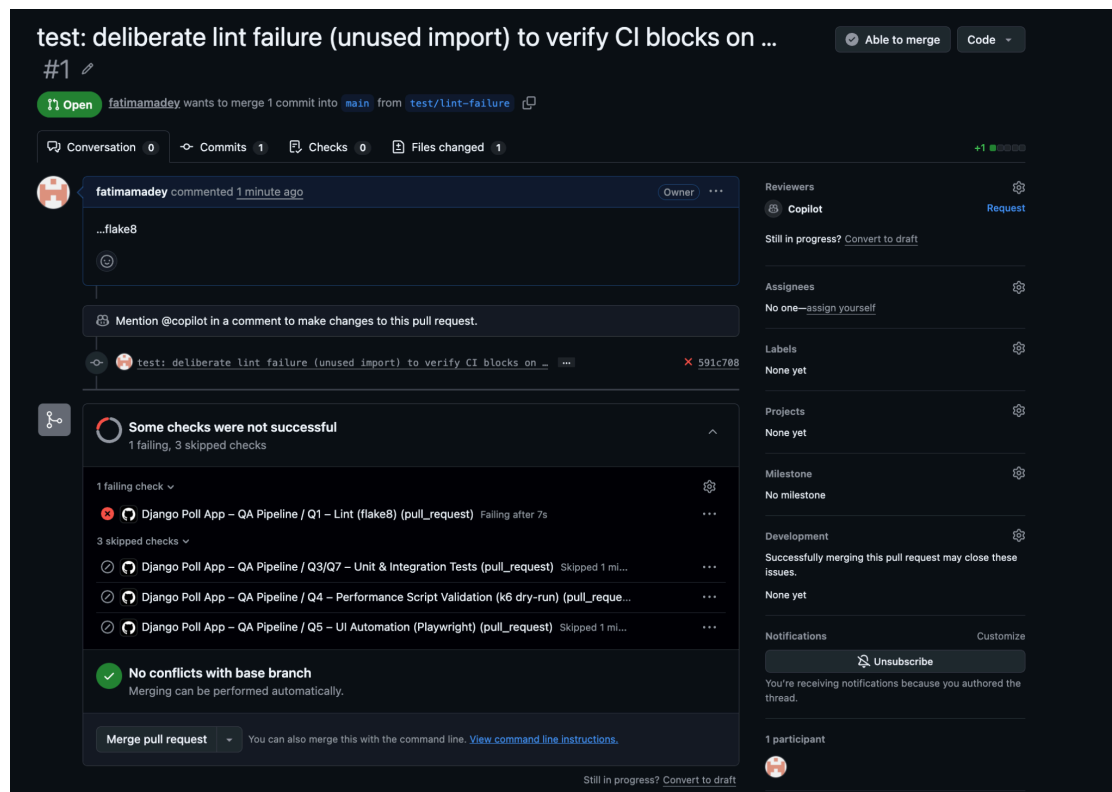
An unused import statement was added to:

None

`app/polls/models.py`

When the pipeline executed:

1. Flake8 detected the unused import.
2. The lint job failed immediately.
3. The remaining jobs were skipped.
4. The pull request could not proceed until the issue was corrected.



Screenshot of failing pipeline run